End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering - Mini-Project Report

Nikunj Mahajan (IMT2023068)

GitHub repository	https://github.com/nikunj169/calculator-devops
Docker Hub image	https://hub.docker.com/r/niki9937/calculator-devops (public, tags *:latest*, *:<build number>*)
Stack	Java 21, Spring Boot 3.5 (Web + Actuator), Maven, JUnit 5 / MockMvc, Git/GitHub, Jenkins LTS (declarative pipeline), Docker (multi-stage), Docker Compose, Nginx

1. Overview

The project is a small Calculator REST API wrapped in a fully automated delivery pipeline. A change merged into **main** on GitHub is detected by Jenkins, compiled, tested, packaged into a Docker image tagged with the Jenkins build number, pushed to Docker Hub and rolled out with Docker Compose behind an Nginx reverse proxy. The pipeline then checks the deployment's health, runs a smoke test, and emails the build result. No step needs manual intervention; build #1 was triggered by Poll SCM after the pull requests were merged and finished green in about four minutes.

2. Pipeline architecture

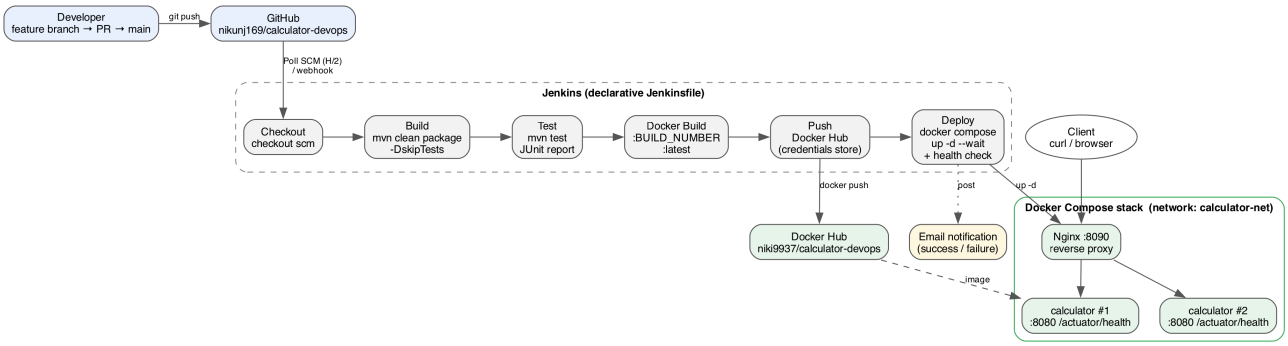


Figure 1 - Flow from a Git push through the six Jenkins stages to Docker Hub and the Compose deployment.

3. Application

Endpoints: `GET /api/v1/{add,subtract,multiply,divide}?a=&b=` return `{"operation":"add","a":2,"b":3,"result":5}`; `/actuator/health` reports UP/DOWN. Arithmetic lives in a plain *CalculatorService*; the controller only validates input and wraps results. A *@RestControllerAdvice* maps missing parameters, non-numeric values, NaN/Infinity and division by zero to HTTP 400 with a structured body (timestamp, status, error, message, path). Whole-number results are rendered as integers so the JSON matches the specification (5, not 5.0).

Tests (30, all passing): parameterised JUnit 5 unit tests for every operation, including divide-by-zero and 0/0; a *@SpringBootTest* + MockMvc integration suite that calls each real endpoint, every error path and the Actuator health/info endpoints. `mvn test` writes Surefire JUnit XML, which Jenkins publishes.

4. Pipeline stages

Stage	What it does
Trigger	`githubPush()` for a GitHub webhook, and `pollSCM('H/2 * * * *')` as the working trigger, since this Jenkins runs on localhost and GitHub cannot reach it. `disableConcurrentBuilds` and a 30-minute timeout keep runs from overlapping.
Checkout	`checkout scm` pulls the commit that triggered the build and logs it.
Build	`mvn -B clean package -DskipTests` produces *target/calculator.jar*.
Test	`mvn -B test`; a non-zero exit fails the stage and the build. `junit 'target/surefire-reports/*.xml'` runs in *post { always }*, so results are published even on failure.
Docker Build	Builds the multi-stage image and tags it *niki9937/calculator-devops:<BUILD_NUMBER>* and *:latest*.

Stage	What it does
Push	Binds the Jenkins credential *dockerhub-credentials* via `credentials()` (masked in logs), logs in with `--password-stdin` using a throwaway `DOCKER_CONFIG`, pushes both tags, logs out and deletes the temporary config.
Deploy	`docker compose up -d --remove-orphans --wait` with `IMAGE_TAG=BUILD_NUMBER`, then polls */actuator/health* through Nginx and runs a smoke request; on failure it prints container logs and fails the build.
Post	`emailext` sends an HTML success mail or a failure mail with the console log attached; dangling images are pruned.

Idempotency: the Compose project name is fixed (*calculator-devops*), so every run updates the same stack. `up -d` only recreates containers whose image or config changed, and a re-run of the same commit leaves the running deployment alone. **Secrets:** no credentials live in the repository; *.gitignore* excludes *target/*, IDE files, *.env* and key files.

5. Containerisation and deployment

Dockerfile (multi-stage): stage 1 (*maven:3.9-eclipse-temurin-21*) copies *pom.xml* first and runs `dependency:go-offline`, so the dependency layer stays cached until the POM changes, then copies *src/* and packages the JAR. Stage 2 (*eclipse-temurin:21-jre-alpine*, about 95 MB) copies only the JAR, creates and switches to a non-root *app* user, declares `EXPOSE 8080` and a `HEALTHCHECK`. A *.dockerignore* limits the build context to *pom.xml* and *src/*. `docker run -p 8085:8080 niki9937/calculator-devops` is all it takes to run it.

docker-compose.yml: two *calculator* replicas (`deploy.replicas: 2`) and an *nginx:1.27-alpine* reverse proxy published on port 8090, on a dedicated bridge network *calculator-net*. Both services have health checks (the API checks */actuator/health* for *"status":"UP"*), `restart: unless-stopped`, and environment variables (*SERVER_PORT*, *JAVA_TOOL_OPTIONS*, *TZ*). Nginx starts only after the replicas report *service_healthy*. It resolves the service name through Docker DNS on every request, which spreads load across replicas (the *X-Upstream-Addr* header alternates) and follows new container IPs after a redeploy. One `docker compose up -d` starts the whole stack.

6. Git workflow

The application was built in small commits on *main* (skeleton, service + unit tests, controller + validation, integration tests). Containerisation, the Jenkinsfile and the README each went through their own feature branch and pull request (#1-#3), merged with merge commits so the branch history stays visible. PR #4 (deployed build number on */actuator/info*) is the change for the live demo: merging it makes Jenkins build, push and deploy the next build automatically. Pushing this report to main already did the same thing: build #2 was triggered by Poll SCM and redeployed only the API replicas, leaving Nginx running.

7. Challenges faced

Problem	Resolution
***eclipse-temurin:17-jre-alpine* has no arm64 image, so the build failed on Apple Silicon (*no match for platform in manifest*).**	Moved to Java 21 LTS, whose Alpine JRE is multi-arch and matches the JDK Jenkins runs on.
The Nginx container stayed *unhealthy*: *localhost* resolved to ::1 inside Alpine, but Nginx listens on IPv4 only.	Health checks use 127.0.0.1 explicitly.
Nginx caches upstream IPs at startup, so after Jenkins recreated the API containers it would proxy to dead addresses.	Used `resolver 127.0.0.11` with a variable in `proxy_pass`, which forces a DNS lookup per request.
The integration test class, at first named **IT*, was silently skipped by Surefire.	Renamed it to *CalculatorControllerIntegrationTest* so `mvn test` runs it.
Jenkins runs on localhost, so a GitHub webhook can't reach it.	Kept `githubPush()` for a public Jenkins and added Poll SCM every 2 minutes.
`docker login/logout` **in the pipeline would overwrite or wipe the host's Docker Desktop keychain login.**	The Push stage uses a temporary `DOCKER_CONFIG` with no credential helper and deletes it afterwards.
Jenkins started by launchd has no Homebrew PATH (*mvn*, *docker*).	Prepended */opt/homebrew/bin* in the pipeline's `environment`.

Problem	Resolution
No SMTP account was configured in Jenkins.	Pointed Email Extension at a local Mailpit SMTP server (localhost:1025) for the demo. Real delivery only needs Gmail SMTP (smtp.gmail.com:465, SSL, app password) configured in Jenkins; the Jenkinsfile stays the same.

Known limitation: the image is built on the Jenkins host, so it is arm64-only. A `docker buildx --platform linux/amd64,linux/arm64` build would make it multi-arch.

Appendix - Screenshots

Jenkins: calculator-devops #1 — console output (key lines)

```
Started by an SCM change
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] { (Checkout)
[Pipeline] { (Build)
[Pipeline] { (Test)
[2026-10-29T17:32:43.680Z] [INFO] Tests run: 29, Failures: 0, Errors: 0, Skipped: 0
[2026-10-29T17:32:43.859Z] Recording test results
[Pipeline] { (Docker Build)
[2026-10-29T17:32:49.132Z] #21 naming to docker.io/niki9937/calculator-devops:1 done
[2026-10-29T17:32:49.132Z] #21 naming to docker.io/niki9937/calculator-devops:latest done
[Pipeline] { (Push)
[2026-10-29T17:32:52.959Z] Login Succeeded
[2026-10-29T17:35:17.363Z] 1: digest: sha256:98528af39985983bfb0eb86946ebdffd5fa67cfc173f60e19e5e5537c73aad0b size: 856
[2026-10-29T17:35:22.126Z] latest: digest: sha256:98528af39985983bfb0eb86946ebdffd5fa67cfc173f60e19e5e5537c73aad0b size:
[Pipeline] { (Deploy)
[2026-10-29T17:35:36.397Z]  Container calculator-devops-calculator-2  Healthy
[2026-10-29T17:35:36.398Z]  Container calculator-devops-calculator-1  Healthy
[2026-10-29T17:35:36.730Z]  Container calculator-devops-calculator-2  Healthy
[2026-10-29T17:35:36.730Z]  Container calculator-devops-calculator-1  Healthy
[2026-10-29T17:35:47.482Z]  Container calculator-devops-nginx-1  Healthy
[2026-10-29T17:35:48.523Z] + echo 'Health check passed'
[2026-10-29T17:35:48.523Z] Health check passed
[2026-10-29T17:35:48.523Z] {"operation":"add","a":2,"b":3,"result":5}+ echo
[Pipeline] { (Declarative: Post Actions)
[2026-10-29T17:35:49.230Z] Sending email to: nikunjmahajan9937@gmail.com
Finished: SUCCESS
```

Figure 2 - Jenkins build #1 of *calculator-devops* (started by an SCM change): all stages ran, 29 tests passed, both tags pushed, stack healthy, email sent, **Finished: SUCCESS**.

Mailpit (SMTP localhost:1025) — Jenkins build notification

```
From:     Jenkins CI <jenkins@calculator-devops.local>
To:       nikunjmahajan9937@gmail.com
Date:     2026-10-29T23:05:49+05:30
Subject:  SUCCESS: calculator-devops #1
--------------------------------------------------------------------------------
Build *#1* of *calculator-devops* succeeded.

* Commit: 2ae7de625ab4714fcc74d8fd331919b5094e0897
* Image: niki9937/calculator-devops:1
* Deployed at: http://localhost:8090

Open build in Jenkins ( http://localhost:8080/job/calculator-devops/1/ )
```

Figure 3 - Build-status email sent by Jenkins to the student's address, captured by the Mailpit SMTP server.

docker ps — running stack

```
$ docker ps --format "table {{.Names}}  {{.Image}}  {{.Status}} {{.Ports}}"
NAMES                          IMAGE                          STATUS                    PORTS
calculator-devops-nginx-1      nginx:1.27-alpine              Up 5 minutes (healthy)    0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
calculator-devops-calculator-2 niki9937/calculator-devops:1   Up 5 minutes (healthy)    8080/tcp
calculator-devops-calculator-1 niki9937/calculator-devops:1   Up 5 minutes (healthy)    8080/tcp

$ docker compose ps
SERVICE      NAME                           STATUS
calculator   calculator-devops-calculator-1   Up 5 minutes (healthy)
calculator   calculator-devops-calculator-2   Up 5 minutes (healthy)
nginx        calculator-devops-nginx-1        Up 5 minutes (healthy)
```

Figure 4 - `docker ps` after the pipeline deploy: two API replicas running image tag *:1* and Nginx, all healthy.

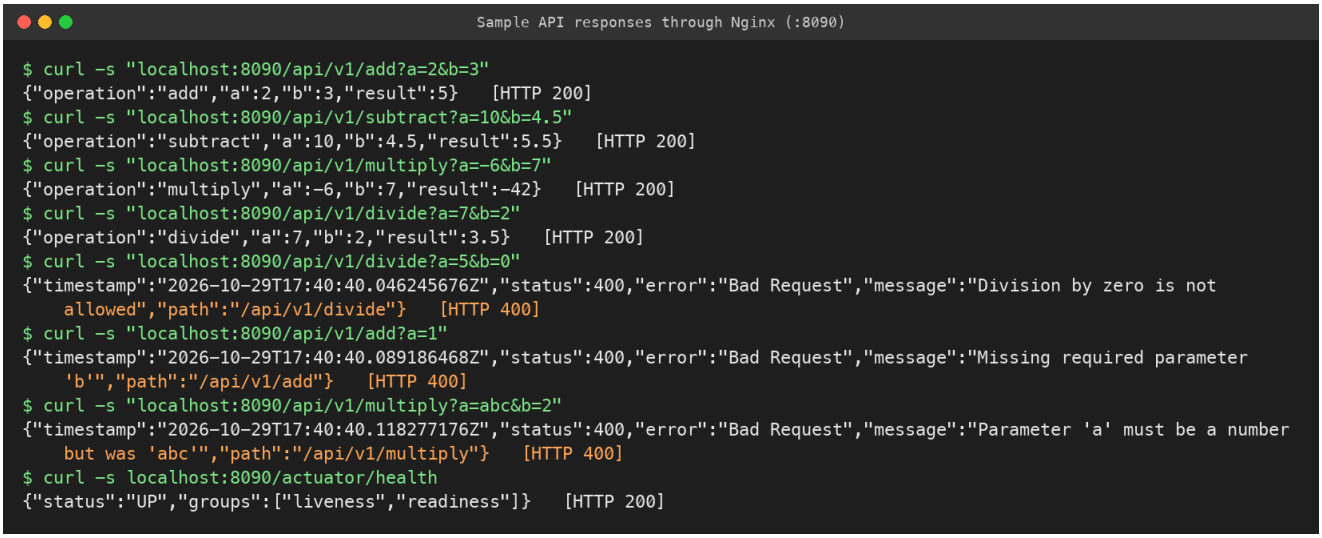


Figure 5 - Sample API responses through Nginx, including the 400 error bodies.

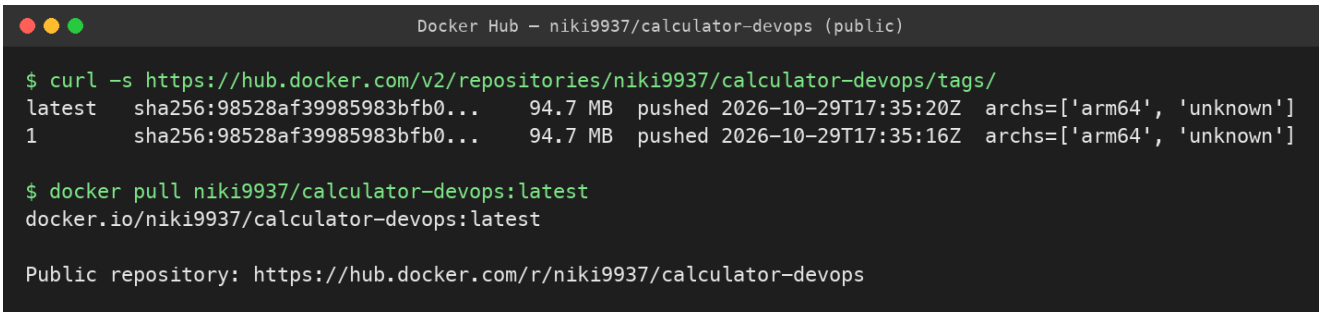


Figure 6 - Public Docker Hub repository with the tags pushed by Jenkins.

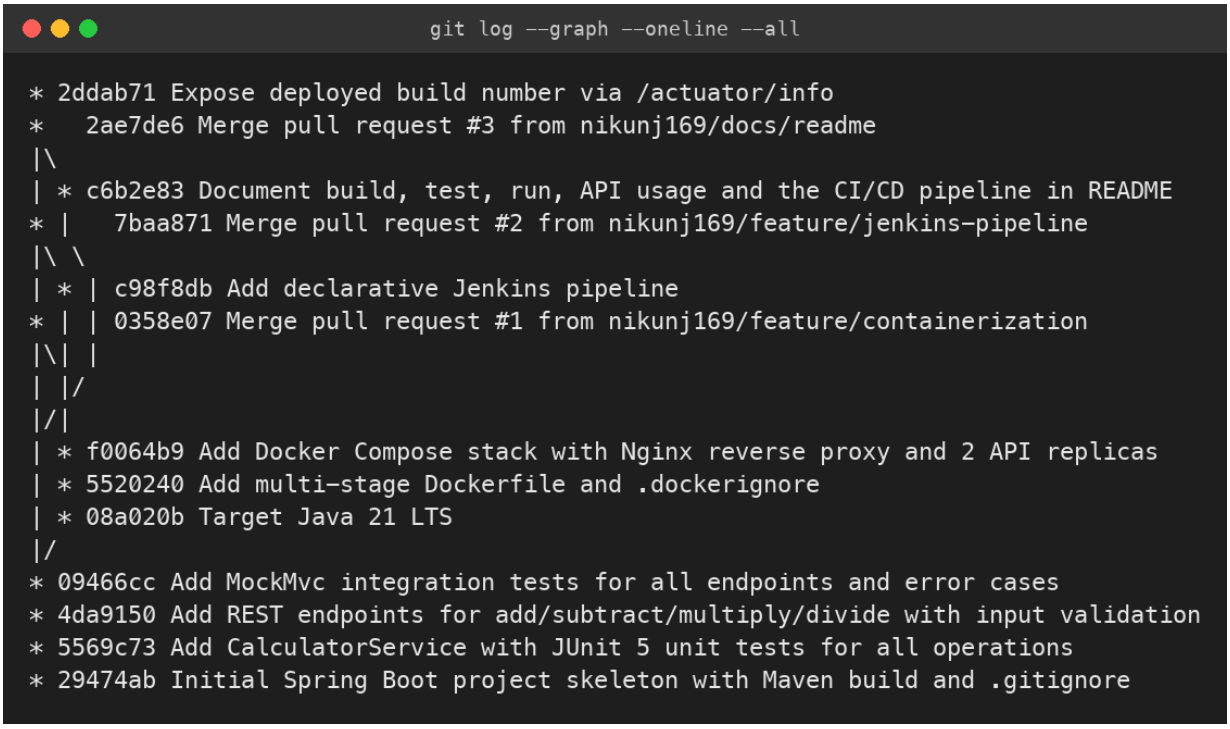


Figure 7 - Commit history with feature branches and merged pull requests.